

مذكرة شاملة في

تحليل البيانات والإحصاء وتطبيقات الذكاء الاصطناعي

Data Analysis, Preprocessing & AI Applications

مذكرة تعليمية متكاملة تغطي:

اكتشاف القيم الشاذة | تنظيف البيانات | الإحصاء الوصفي
البيانات الزمنية | البيانات النصية والصورية | EDA
مشاريع تطبيقية | AI العملية | تطبيقات Python

Outlier Analysis) الفصل الأول :اكتشاف القيم الشاذة

1.1 — الصورة الكاملة Outliers ما هي الـ

تخيل إنك بتحلل رواتب موظفين في شركة، ومعظم الرواتب بين 5,000 و 15,000 جنيه، وفجأة لقيت قيمة 500,000 جنيه في البيانات. هذه قيمة شاذة بعيدة جداً عن باقي البيانات — Outlier القيمة هي اللي بنسميها

بشكل رسمي هو أي نقطة بيانات تتحرف انحرافاً كبيراً عن النمط العام للبيانات. المشكلة مش في القيمة نفسها — المشكلة إن وجودها Outlier الـ ممكن يخلل الموديل اللي هتبنيه ويدّيك نتائج غلط تماماً

حساسية K-Means Clustering و Linear Regression زي Machine Learning ؟ لأن كتير من خوارزميات الـ AI ليه ده مهم للـ. هياخد قرارات غلط على بيانات حقيقية، outliers لو موديلك اتدرب على بيانات فيها. Outliers جداً للـ

Outliers أنواع الـ 1.2

مش نوع واحد، ومهم تعرفهم عشان تعرف تتعامل مع كل نوع صح Outliers الـ

الوصف والمثال	Outlier نوع الـ
قيمة منفردة بعيدة عن الكل — مثل راتب 500,000 وسط رواتب 5,000-15,000	Point Outlier (نقطة شاذة)
قيمة عادية في سياق معين لكن شاذة في سياق آخر — مثل 30 درجة حرارة في الشتاء شاذة لكن في الصيف طبيعية	Contextual Outlier (شاذ سياقياً)
— مجموعة نقاط مع بعض شاذة حتى لو كل نقطة لوحدها طبيعية مثل نبضات قلب سريعة لمدة 30 دقيقة متواصلة	Collective Outlier (مجموعة شاذة)

Outliers أسباب ظهور الـ 1.3

لازم تفهم منين جه — لأن السبب هو اللي بيحدد قرارك، outlier قبل ما تقرر تتعامل مع

- خطأ في القياس أو التسجيل: حد كتب 500,000 بدل 5,000 — خطأ إنساني بسيط
- خطأ في التجميع: البيانات اتجمعت من مصادر مختلفة بوحدات مختلفة
- حقيقي ومش غلط outlier — Black Friday حدث استثنائي حقيقي: مبيعات ضخمة في تغيير في النظام: تحديث في طريقة القياس خلّى القيم تبدو شاذة
- Anomaly Detection احتيال أو تلاعب: معاملة مالية مشبوهة — ده بالذات مفيد في

مثال تطبيقي — أسبقية التحقيق

لو وجدت درجة طالب 200 من 100، فده خطأ تسجيل واضح. أما لو وجدت طالب درجته 99 وكل الطلاب بين 50-70، فده ممكن يكون إطالب موهوب حقيقي — لا تحذفه

من الأبسط للأكثر دقة — Outliers طرق اكتشاف الـ 1.4

IQR (Interquartile Range) طريقة

وفكرتها بسيطة وقوية في نفس الوقت. نقسم البيانات لأربع أرباع، ونشوف الفرق، Data Science هي الطريقة الأكثر استخداماً في الـ IQR الـ بين الربع الثالث والربع الأول.

من المنظور الرياضي

```
الربع الأول ( 25% من البيانات تحته ) Q1 =
الربع الثالث ( 75% من البيانات تحته ) Q3 =
IQR = Q3 - Q1
الحد الأدنى (Lower Bound) = Q1 - 1.5 × IQR
الحد الأقصى (Upper Bound) = Q3 + 1.5 × IQR
Outlier = أي قيمة خارج هذين الحدين
```

عشان يغطي تقريباً 99.3% من البيانات اللي بتتبع التوزيع الطبيعي. لو استخدمت 3 بدل John Tukey ليه 1.5 بالذات؟ ده اختار إحصائي. هتكتشف فقط الشواذ الشديدة جداً 1.5.

مثال عملي: رواتب [4000, 5000, 6000, 7000, 8000, 9000, 10000, 50000]

```
Q1 = 5500, Q3 = 9500, IQR = 4000
(مفیش راتب سالب، فمش هياثر) Lower = 5500 - 6000 = -500
Upper = 9500 + 6000 = 15500
15500 > 50000 → Outlier ✓
```

IQR بطريقة Outliers لاكتشاف الـ Python كود

```
import pandas as pd
import numpy as np

def detect_outliers_iqr(data, column):
    Q1 = data[column].quantile(0.25) # الربع الأول
    Q3 = data[column].quantile(0.75) # الربع الثالث
    IQR = Q3 - Q1 # المدى الربيعي

    lower_bound = Q1 - 1.5 * IQR # الحد الأدنى
    upper_bound = Q3 + 1.5 * IQR # الحد الأقصى

    # نرجع البيانات الشاذة
    outliers = data[(data[column] < lower_bound) |
                    (data[column] > upper_bound)]
    return outliers, lower_bound, upper_bound
```

Z-Score طريقة

بيقيسلك 'كام انحراف معياري' القيمة دي بعيدة عن المتوسط. ده بيفيدك لما تيجي تقارن بيانات من مقاييس مختلفة Z-Score الـ

$$Z = (X - \mu) / \sigma$$

الانحراف المعياري σ ، المتوسط μ ، القيمة X : حيث
Outlier $\leftarrow |Z| > 3$ القاعدة: لو

اللي بره ده نادر جداً وبنعتبره شاذ $[\mu - 3\sigma, \mu + 3\sigma]$ الفكرة الحدسية: في أي توزيع طبيعي، 99.7% من البيانات بتقع في المجال
أفضل لأنه مش بيفترض IQR لو البيانات مش موزعة طبيعياً، الـ (Bell Curve) ؟ لما بياناتك تتبع التوزيع الطبيعي Z-Score متى تستخدم
شكل معين للتوزيع

```
from scipy import stats

def detect_outliers_zscore(data, column, threshold=3):
    z_scores = np.abs(stats.zscore(data[column]))
    outliers = data[z_scores > threshold]
    return outliers
```

Modified Z-Score طريقة

كبير جداً، هيرفع outlier يعني لو عندك Outliers! الكلاسيكي إن المتوسط والانحراف المعياري أنفسهم بيتأثروا بالـ Z-Score المشكلة في
بدل ما يكشفه outlier المتوسط ويوسع الانحراف المعياري، وبالتالي قد يخفي الـ
MAD (Median Absolute Deviation) والانحراف المعياري بـ Median الحل؟ نستبدل المتوسط بـ

$$MAD = \text{median}(|X_i - \text{median}(X)|)$$

$$\text{Modified Z-Score} = 0.6745 \times (X - \text{median}) / MAD$$

Outlier $\leftarrow |\text{Modified Z-Score}| > 3.5$ القاعدة: لو

أقوى وأكثر مقاومة Modified Z-Score الكلاسيكي للبيانات الطبيعية. الـ Z-Score الـ 0.6745 ده عامل تصحيح يخلي النتيجة متوافقة مع الـ
من الطريقة الكلاسيكية Outliers للـ

فكرة العزل — Isolation Concept

أسهل في العزل من النقاط الطبيعية. تخيل إنك بتعزل نقاط Outliers خوارزمية ذكية جداً مبنية على فكرة بسيطة: الـ Isolation Forest الـ
بشكل عشوائي — النقطة الشاذة المعزولة عادةً بتت عزل في خطوات أقل بكثير من النقطة الطبيعية اللي في وسط التجمع
الفكرة الحدسية: لو عندك 100 تفاحة ورمانة واحدة وقلت لحد 'عزل الرمانة' — هيعزلها بسرعة لأنها مختلفة. نفس الفكرة بالضبط

```
from sklearn.ensemble import IsolationForest

model = IsolationForest(contamination=0.05) # 5% outliers
data['anomaly'] = model.fit_predict(data[['salary']])
# القيمة 1 تعني طبيعي، outlier القيمة -1 تعني
outliers = data[data['anomaly'] == -1]
```

الاكتشاف البصري — Visual Detection

بشكل سريع Outliers أحياناً أفضل أداة عندك هي عينيك! الرسوم البيانية بتكشف أنماط الـ

نوع الرسم

ما يكشفه

Box Plot (المربع والشوارب)	والنقاط خارج الحدود كنقاط منفردة Q1, Q3, IQR يظهر
Scatter Plot	يكشف القيم البعيدة عن التجمعات الرئيسية
Histogram	(long tails) يظهر التوزيع والذيل الطويلة
Violin Plot	وتوزيع الكثافة Box Plot مزيج بين

```

import matplotlib.pyplot as plt
import seaborn as sns

# Box Plot
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='salary')
plt.title('Outliers توزيع الرواتب مع الـ')
plt.show()

```

AI Models على Outliers تأثير الـ 1.5

AI؟ في سياق الـ Outliers ده القلب الحقيقي للموضوع — ليه إحنا أصلاً بنهتم بالـ

الموديل	Outliers كيف تأثره الـ
Linear Regression	بيشد خط الانحدار ناحيته بشكل كبير — نتائج غلط تماماً
K-Means Clustering	سيئ clustering — بيأثر على مراكز التجمعات
Neural Networks	كبيرة وعدم استقرار في التدريب gradients ممكن يسبب
Decision Trees	أقل تأثر نسبياً لأنها بتقسم البيانات بدل حساب متوسطات
SVM	Support Vectors يأثر على اختيار الـ

استراتيجيات 3 — Outliers التعامل مع الـ 1.6

وطبيعة مشروك Outlier عندك 3 خيارات رئيسية. اختيارك بيعتمد على السبب وراء الـ Outliers، بعد ما اكتشفت الـ

(Deletion) الاستراتيجية 1: الحذف

صغير مقارنة بحجم البيانات outliers خطأ واضح (غلطة إدخال بيانات (وعدد الـ outlier من البيانات مناسب لما يكون الـ outlier لما تحذف الـ حقيقي وبيمثل معلومة مهمة، أو لما عندك بيانات قليلة أصلاً outlier لا تستخدم الحذف لما الـ

```

# بعد اكتشافهم بـ outliers حذف الـ
df_clean = df[(df['salary'] >= lower_bound) &
              (df['salary'] <= upper_bound)]

```

(Clipping) الاستراتيجية 2: القص

بدل الحذف، بنستبدل القيم الشاذة بالحد الأقصى أو الأدنى المسموح به. يعني لو عندك قيمة 500,000 والحد الأقصى 15,500، بنخليها 15,500

مهم معرفة وجوده لكن مش قيمته الحقيقية outlier متى تستخدمه؟ لما البيانات محدودة ومش تقدر تحذف، أو لما الـ

```
# Clipping - القيم عند الحدود
df['salary_clipped'] = df['salary'].clip(
    lower=lower_bound,
    upper=upper_bound
)
```

Winsorization: الاستراتيجية 3

بنسبة 5% معناها إن أقل 5% وأعلى Winsorize لكن أكثر مرونة — بتحدد النسبة المئوية اللي تقطعها من الطرفين. مثلاً Clipping شبيهة بالـ
من البيانات بتتستبدل بالقيمة عند النسبة المئوية المحددة 5%

```
from scipy.stats.mstats import winsorize

# Winsorize - تعديل 5% من كل طرف
df['salary_wins'] = winsorize(df['salary'], limits=[0.05, 0.05])
```

الفصل الثاني: تنظيف البيانات وتجهيزها (Data Cleaning & Preprocessing)

الصورة الكاملة

يقولون إن 80% من وقت المشروع يبتصرف في تنظيف البيانات — مش في بناء الموديل Data Scientists بيانات نظيفة = موديل ذكي. الـ 'Garbage In, Garbage Out'. لأن أي موديل مهما كان ذكياً، لو اتدرب على بيانات مزعقة، هيطلع نتائج مزعقة

البيانات المفقودة — 2.1 Missing Data

تخيل إنك بتحلل بيانات مرضى وعندهك عمود 'ضغط الدم' فيه قيم فاضية — ليه؟ Data Science القيم المفقودة هي إحدى أكبر التحديات في الـ المريض رفض القياس؟ جهاز اتعطل؟ الممرض نسي يسجل؟

أنواع القيم المفقودة — مهم جداً للقرار

الشرح والمثال	النوع
الغياب عشوائي تماماً ومش مرتبط بأي شيء — مثل بيانات اتمسحت بسبب خلل تقني	MCAR (Missing Completely At Random)
الغياب مرتبط ببيانات موجودة — مثل النساء اللي رفضن يسجلن وزنهم أكثر من الرجال	MAR (Missing At Random)
الغياب مرتبط بالقيمة المفقودة نفسها — مثل أصحاب الدخل العالي اللي رفضوا يكشفوا دخلهم	MNAR (Missing Not At Random)

Bias. الأصعب لأن حذف البيانات هيعمل MNAR، الأبسط في التعامل MCAR. ليه النوع مهم؟ لأنه يحدد الطريقة الصح للتعامل

2.2 Imputation طرق التعامل مع القيم المفقودة

معناه 'تعينة' القيم المفقودة بقيم محسوبة. فكرة في المنطق: بدل ما تحذف صف كامل عشان فيه قيمة واحدة ناقصة، تملأه بأفضل Imputation الـ تخمين ممكن.

التعينة بالمتوسط — Mean Imputation

مثل الأطوال أو درجات — (Symmetric) الأبسط والأسرع: بنملا القيم الفاضية بمتوسط العمود. مناسب للبيانات الرقمية الموزعة توزيعاً متماثلاً الحرارة.

في البيانات (Variance) المتوسط هيتأثر وهيديك قيمة غلط. وبتقلل التباين، Outliers المشكلة: لو البيانات فيها

```
df['height'].fillna(df['height'].mean(), inplace=True)
```

التعبئة بالوسيط — Median Imputation

لأن الوسيط مش بيتأثر بالقيم الكبيرة جداً. مثالي للبيانات المنحرفة Outliers بدل المتوسط، نستخدم الوسيط (القيمة الوسطية). (أقوى مقاومة للـ مثل الرواتب والأسعار (Skewed))

```
df['salary'].fillna(df['salary'].median(), inplace=True)
```

التعبئة بالمنوال — Mode Imputation

زني النوع الاجتماعي أو فئة المنتج. ينملاً بالقيمة الأكثر تكراراً. مشكلته إنه بيزيد تحيز ناحية الفئة الأكثر شيوعاً (Categorical) للبيانات الفئوية

```
df['gender'].fillna(df['gender'].mode()[0], inplace=True)
```

للبيانات الزمنية — Forward Fill و Backward Fill

ببملاً بأول قيمة معروفة Backward Fill ببملاً القيمة الفاضية بأخر قيمة معروفة قبلها، والـ Forward Fill الـ Time Series في بيانات بعدها. منطق الفكرة: درجة الحرارة امبارح 25 درجة، ودرجة الحرارة قبلها 24 — فالقيمة الفاضية يرجح إنها في نفس النطاق

```
df['temperature'].fillna(method='ffill', inplace=True) # Forward Fill
df['temperature'].fillna(method='bfill', inplace=True) # Backward Fill
```

البيانات المكررة — 2.3 Duplicate Data

أو يزن مثال Overfitting بيانات مكررة بتعني إن نفس الصف موجود أكثر من مرة. ده بيخلي الموديل يتعلم من نفس المثال أكثر من مرة ويعمل معين أكثر من اللازم

```
# اكتشاف المكرر
duplicates = df[df.duplicated()]
print(f'عدد الصفوف المكررة: {duplicates.shape[0]}')

# حذف المكرر والاحتفاظ بأول ظهور
df = df.drop_duplicates(keep='first')
```

2.4 Noisy Data و Inconsistent Data

هي بيانات متضاربة Inconsistent Data هي بيانات فيها أخطاء عشوائية أو تشويش — مثل جهاز قياس غير دقيق. الـ Noisy Data مثل نفس الشخص عنده تاريخ ميلاد مختلف في مكانين —

لـ Business Rules والـ Data Validation و (Moving Average مثل) Noisy Data للـ Smoothing: طرق التعامل Inconsistent Data.

تحويل البيانات الفئوية — 2.5 Encoding

الخوارزميات الرياضية تتعامل مع أرقام مشصوص. لما عندك عمود 'لون' فيه 'أحمر، أزرق، أخضر' — لازم تحوله لأرقام. بس كيف؟ ده بيعتمد على نوع البيانات الفئوية.

One-Hot Encoding

مثل الألوان أو الجنسيات. بنحول كل فئة لعمود جديد بقيمة 0 أو 1. مثال: أحمر، أزرق، أخضر 'بتصبح' — (Nominal) لما الفئات مش مترتبة [is_red, is_blue, is_green] 3 أعمدة.

Curse of Dimensionality المشكلة: لو عندك فئة بها 100 قيمة ممكنة، هتضيف 100 عمود إده اللي بنسميه

```
# One-Hot Encoding
df = pd.get_dummies(df, columns=['color'], prefix='color')
# sklearn أو باستخدام
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder(sparse=False, drop='first') # drop='first' يتجنب التكرار
encoded = encoder.fit_transform(df[['color']])
```

Label Encoding

مثل 'صغير، متوسط، كبير'. بنحول كل فئة لرقم متسلسل. الترتيب مهم! 'صغير=0، متوسط=1، كبير=2' منطقي — (Ordinal) لما الفئات مترتبة لأن $0 < 1 < 2$.

(بدون ترتيب) — لأن الخوارزمية هتفتكر إن 'أحمر=2' أكبر من 'أزرق=1' وده مش Nominal مع بيانات Label Encoding لا تستخدم منطقي.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df['size_encoded'] = le.fit_transform(df['size']) # 0→صغير، 1→متوسط، 2→كبير
```

تحويل شكل البيانات — 2.6 Data Transformation

(Positively Skewed) أحياناً البيانات مش في الشكل المناسب للتحليل أو للموديل. مثلاً بيانات الرواتب أو الأسعار عادةً بتكون منحرفة جداً لليمين فيه أقلية بتأخذ رواتب ضخمة جداً. الخوارزميات بتشتغل أحسن مع بيانات توزيعها متماثل —

Log Transformation

بيقرب القيم الكبيرة من بعض ويفرق القيم الصغيرة log على البيانات لتقليل الانحراف. الـ Logarithm بنطبق

```
df['log_salary'] = np.log(df['salary'] + 1) # log(0) عشان نتجنب +1
# [6.9, 7.6, 9.2, 11.5] — توزيع أكثر تقارباً log مثال: الرواتب [1000, 2000, 10000, 100000] بعد
```

Power Transformation (Box-Cox)

اللي بتخلي البيانات أقرب للتوزيع الطبيعي (λ) لأنه بيختار تلقائياً أفضل قيمة لـ Log تحويل أكثر مرونة من الـ

```
from sklearn.preprocessing import PowerTransformer
pt = PowerTransformer(method='box-cox') # يتطلب قيم موجبة فقط
df['transformed'] = pt.fit_transform(df[['salary']])
```

الفصل الثالث: الإحصاء الوصفي متعدد المتغيرات

Multivariate: الصورة الكاملة — 3.1

بنسأل أسئلة أعمق، Multivariate كنت بتسأل أسئلة زي 'ما متوسط الراتب؟'، 'لما بنيجي لك' (Univariate) لما درست إحصاء أحادي المتغير. 'هل في علاقة بين الراتب والخبرة؟' و'كيف بتتغير هذه العلاقة مع اختلاف القطاع؟'

مبني — لأن أي موديل بيتعلم من العلاقات بين المتغيرات Machine Learning هي الأساس اللي عليه كل الـ Multivariate Statistics الـ المختلفة.

التوزيع المشترك — Joint Distribution

التوزيع المشترك بيصف احتمالية حدوث قيم لمتغيرين أو أكثر في نفس الوقت. مثلاً: بما احتمالية إن الراتب > 10,000 والخبرة > 5 سنين في نفس الوقت؟

والـ Features بين الـ Joint Distribution في الأساس بيحاول يتعلم الـ Regression أو Classification ليه مهم؟ لأن أي موديل Target.

التوزيع الهامشي — Marginal Distribution

لهذا Marginal Distribution لعمود واحد، ده هو الـ Histogram هو توزيع متغير واحد بغض النظر عن المتغيرات التانية. يعني لما بتعمل المتغير.

التوزيع الشرطي — Conditional Distribution

Naive توزيع متغير معين بشرط إن متغير تاني له قيمة محددة. مثال: ما توزيع الراتب بشرط إن الخبرة = 5 سنين؟ ده أساسي في خوارزمية Bayes وفي فهم العلاقات بين المتغيرات.

مصفوفة التباين — 3.2 Covariance Matrix

في المصفوفة بتقولك كيف يتحرك المتغير (i,j) هي ملخص للعلاقات بين كل المتغيرات في مجموعة بياناتك. كل خلية Covariance Matrix الـ j مع المتغير i .

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)]$$

المتغيران يتحركان في نفس الاتجاه: $\text{Cov} > 0$ لو
المتغيران يتحركان في اتجاهين معاكسين: $\text{Cov} < 0$ لو
لا توجد علاقة خطية: $\text{Cov} \approx 0$ لو

أساسية في Covariance Matrix الـ

- لتقليل الأبعاد — PCA (Principal Component Analysis)
- قياس المسافة بين نقاط — Mahalanobis Distance
- في التمويل — Portfolio Optimization

```
cov_matrix = df.cov()
import seaborn as sns
sns.heatmap(cov_matrix, annot=True, cmap='coolwarm')
```

مقاييس المسافة — 3.3 Distance Metrics

تتبع لها K-Means و KNN مهمة جداً — موديلات زي Feature Space المسافة 'بين نقطتين في الـ' Machine Learning في الـ بشكل كامل.

المسافة الإقليدية — Euclidean Distance

لكثير من الخوارزميات default المسافة المستقيمة بين نقطتين — زي ما بنقيس المسافة بالمسطرة في الواقع. ده الـ

```
d = sqrt((x1-x2)2 + (y1-y2)2 + ...)
```

Normalization: بـقيم 0-1000، الثاني هيسطر على المسافة. الحل another بـقيم 0-1 و feature مشكلتها: بتتأثر بمقياس البيانات. لو عندك قبل الحساب

مسافة مانهاتن — Manhattan Distance

مجموع الفروق المطلقة في كل اتجاه — سميت كده لأنها زي المشي في شوارع مانهاتن (بتيجي يمين/شمال/فوق/تحت بس مش قطري). (أقل تأثراً من الإقليدية Outliers بالـ)

```
d = |x1-x2| + |y1-y2| + ...
```

تشابه الجيب التمام — Cosine Similarity

مش بتقيس المسافة، بتقيس الزاوية بين متجهين. القيمة بين -1 و 1. 1 = نفس الاتجاه، 0 = متعامدان، -1 = اتجاهين معاكسين. بتقارن 'اتجاه' محتوى الوثيقتين Cosine Similarity لو عندك وثيقتين، كمية الكلمات مختلفة، لكن (NLP) أهم استخداماتها: تحليل النصوص. عالي حتى لو إحدهما أطول من الثانية Cosine Similarity وثيقتان عن 'كرة القدم' هيكون بينهما

```
from sklearn.metrics.pairwise import cosine_similarity
similarity = cosine_similarity(doc_matrix)
```

Curse of Dimensionality والـ 3.4 Feature Space

كل نقطة بيانات في فضاء ثلاثي الأبعاد، features هو الفضاء الرياضي اللي فيه كل نقطة بيانات موجودة. لو عندك 3 Feature Space الـ! في فضاء من 100 بُعد، feature لو عندك 100

مثال: لو عندك 100 نقطة في (sparse) 'كلما زاد عدد الأبعاد، البيانات تصبح أكثر 'تفرقاً': لعنة الأبعاد (Curse of Dimensionality) لكل نقطة وحيدة ومعزولة — دكل شيء قريب. نفس 100 نقطة في فضاء 100 — (1D) خط مستقيم

AI: تأثيرها على الـ

- يصبح غير فعال لأن كل النقاط تقريباً نفس المسافة من بعضها KNN
 - احتياجنا لبيانات يتضاعف أسياً مع زيادة الأبعاد
 - Overfitting بتزيد احتمالية الـ
- (PCA, t-SNE, UMAP) تقليل الأبعاد مع الحفاظ على أكبر قدر من المعلومات — Dimensionality Reduction: الحل

(EDA) الفصل الرابع: تحليل البيانات الاستكشافي

4.1 ما هو EDA؟ ولماذا مهم؟

هي المرحلة التي نتعرف فيها على بياناتك قبل ما تبني أي موديل. تخيل إنك بتسافر لبلد جديدة — مش هتروح وتسلق مباشرة، أول شيء EDA الـ بالضبط كده بالنسبة للبيانات EDA. هتعمله إنك تتعرف على المكان، تشوف الخريطة، تفهم الطقس، وتسال الناس مهمة كانت ممكن تتغير شكل مشروعك patterns خاطئة، أو تقوت features ممكن تبني موديل على بيانات مش مناسبة، تختار EDA بدون كله.

4.2 رحلة متكاملة — EDA خطوات

(Data Understanding) الخطوة 1: فهم البيانات

أول ما تاخذ البيانات، لازم تعرف إيه اللي في إيدك

```
df = pd.read_csv('data.csv')

# الشكل العام
print(df.shape)      # كام صف وكام عمود؟
print(df.dtypes)     # نوع كل عمود
print(df.head())     # أول 5 صفوف
print(df.describe()) # ملخص إحصائي
print(df.isnull().sum()) # القيم المفقودة
```

Feature Distribution الخطوة 2: تحليل

بتحلل توزيعها، feature لكل

- (Numerical): Histogram + Box Plot + Mean/Median/Std للأرقام
- النسب المئوية + Bar Chart + Value Counts (Categorical): للفئات

```
# توزيع البيانات الرقمية
df['age'].hist(bins=30)
print(f'Skewness: {df["age"].skew():.2f}') # - = يسار، + = يمين

# البيانات الفئوية
df['category'].value_counts().plot(kind='bar')
```

الخطوة 3: اكتشاف العلاقات

target والـ features لوحدها، نتيجي المرحلة الأهم: فهم العلاقات بين الـ feature بعد ما فهمت كل

```

# Correlation Matrix - العلاقات بين الأرقام
corr = df.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', center=0)

# Scatter Plot - علاقة بين متغيرين
sns.scatterplot(data=df, x='experience', y='salary')

# Pair Plot - كل المتغيرات مع بعض
sns.pairplot(df, hue='target')

```

كاملة EDA Pipeline: الخطوة 4

منظمة pipeline لـ EDA الحقيقية، بنحوّل الـ AI في مشاريع الـ

```

def full_eda(df, target_col):
    print('='*50)
    print('1. معلومات عامة')
    print(df.info())

    print('\n2. القيم المفقودة')
    missing = df.isnull().sum()
    print(missing[missing > 0])

    print('\n3. التوزيع الإحصائي')
    print(df.describe())

    print('\n4. Target الارتباط مع الـ')
    corr_with_target = df.corr()[target_col].sort_values(ascending=False)
    print(corr_with_target)

    return corr_with_target

```

الفصل الخامس: الإحصاء الوصفي للبيانات الزمنية

ما هي البيانات الزمنية؟ 5.1

هي أي بيانات اتسجلت على مدار فترة زمنية بترتيب متسلسل. أسعار الأسهم كل يوم، درجات الحرارة كل ساعة، مبيعات شركة Time Series كل شهر — كلها.

إن ترتيب الوقت مهم جداً! إمّش ممكن تخلط الصفوف عشوائياً زي البيانات العادية Time Series الخاصة المميزة للـ

Time Series مكونات 5.2

المكوّن	الشرح والمثال
Trend (الاتجاه)	التغير التدريجي على المدى الطويل — مثل نمو مبيعات شركة ناشئة على 5 سنين
Seasonality (الموسمية)	أنماط تتكرر بشكل منتظم — مثل ارتفاع مبيعات الملابس في الشتاء كل سنة
Cyclic Patterns (الدورات)	أنماط تتكرر لكن بفترات غير منتظمة — مثل دورات الاقتصاد (انتعاش/ركود)
Noise (الضوضاء)	التقلبات العشوائية الغير قابلة للتفسير — ما يتبقى بعد إزالة الثلاثة الأولى

Time Series المفاهيم الأساسية في تحليل 5.3

المتوسط المتحرك — Moving Average

بنافذة 7 أيام معناها متوسط آخر 7 Moving Average بدل إنك تتعامل مع كل نقطة بيانات منفردة، بتحسب متوسط نافذة زمنية متحركة. مثلاً Trend. وبتكشف الـ Noise أيام لكل نقطة. يتخفف الـ

```
df['MA_7'] = df['sales'].rolling(window=7).mean() # أيام 7
df['MA_30'] = df['sales'].rolling(window=30).mean() # يوم 30
```

إحصاءات متحركة — Rolling Statistics

Time Series في الـ Anomalies متحرك. ده مفيد جداً في اكتشاف الـ Max أو Min أو Std نفس الفكرة لكن مش بس المتوسط — ممكن تحسب Series.

```
df['rolling_std'] = df['sales'].rolling(window=7).std()
df['rolling_min'] = df['sales'].rolling(window=7).min()
```



```
df['rolling_max'] = df['sales'].rolling(window=7).max()
```

مميزات التأخير — Lag Features

في نماذج Features لمبيعات اليوم هي مبيعات امبارح. دي من أقوى الـ Lag-1 هي قيم المتغير في أوقات سابقة. مثلاً Lag Features الـ التنبؤ لأن الماضي غالباً بيتنبأ بالمستقبل.

```
df['sales_lag1'] = df['sales'].shift(1) # مبيعات أمس  
df['sales_lag7'] = df['sales'].shift(7) # مبيعات قبل أسبوع  
df['sales_lag30'] = df['sales'].shift(30) # مبيعات الشهر الماضي
```

الفصل السادس: الإحصاء الوصفي للبيانات النصية والصورية

6.1 تحليل البيانات النصية (Text Data)

النصوص مش أرقام — إزاي نطبق الإحصاء عليها؟ الإجابة: نحول النصوص لأرقام بطرق ذكية تحافظ على المعنى.

تكرار الكلمات — Word Frequency

أبسط تحليل للنصوص: إيه أكثر الكلمات ظهوراً؟ ده بيديك فكرة عن المحتوى الأساسي للنصوص.

```
from collections import Counter
words = ' '.join(df['text']).lower().split()
word_freq = Counter(words)
print(word_freq.most_common(20)) # أكثر 20 كلمة
```

N-Grams

كل 3 كلمات. ده بيحافظ على Trigrams. كل كلمتين متجاورتين = Bigrams. كلمات N بدل ما نحلل كلمات منفردة، بنحلل تسلسلات من منفصلين 'New' و 'York' أفضل من Bigram كـ 'New York' السياق — مثلاً

```
from sklearn.feature_extraction.text import CountVectorizer
bigram_vectorizer = CountVectorizer(ngram_range=(2, 2))
bigrams = bigram_vectorizer.fit_transform(df['text'])
```

أذكى من مجرد العدد — TF-IDF

— هو مقياس لأهمية الكلمة (Inverse Document Frequency) IDF هو تكرار كلمة في وثيقة. الـ TF (Term Frequency) الـ الكلمات الشائعة في كل الوثائق زي 'و' و'في' مش مهمة

```
TF-IDF = TF × IDF
TF(t,d) = عدد مرات ظهور الكلمة t في الوثيقة d
IDF(t) = log(عدد الوثائق الكلي / عدد الوثائق التي تحتوي t)
```

بيديك وزن عالي للكلمات اللي بتظهر كتير في وثيقة معينة لكن نادرة في الوثائق الأخرى — دي الكلمات المميزة للوثيقة TF-IDF الـ

```
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer(max_features=1000)
X = tfidf.fit_transform(df['text'])
```

6.2 تحليل بيانات الصور (Image Data)

قيمة بين 0 و 255. تحليل توزيع هذه القيم بيديك معلومات كثيرة عن الصورة pixel الصورة في النهاية مصفوفة من الأرقام — كل

Pixel Distribution و Color Histograms

= للصورة يظهر توزيع قيم البيكسل. صورة مظلمة =قيم متجمعة قرب 0. صورة فاتحة =قيم قرب 255. صورة متباينة Histogram الـ توزيع منتشر.

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread('image.jpg')

# Histogram لكل قناة لون
for i, color in enumerate(['Blue', 'Green', 'Red']):
    hist = cv2.calcHist([img], [i], None, [256], [0, 256])
    plt.plot(hist, color=color.lower(), label=color)
plt.legend()
```

Mean & Std Normalization للصور

بتطرح المتوسط وتقسّم على الانحراف المعياري. ده بيخلي قيم البيكسل حول — Normalization لازم تعمل AI، قبل إدخال الصور لنموذج الصفر ويسرّع التعلم.

```
mean = img.mean() # متوسط قيم البيكسل
std = img.std() # الانحراف المعياري
img_normalized = (img - mean) / std
```

:(المعروفة ImageNet normalization) Deep Learning في الـ

```
mean = [0.485, 0.456, 0.406] # لكل قناة (R, G, B)
std = [0.229, 0.224, 0.225]
```

Python الفصل السابع: الإحصاء الوصفي باستخدام

التقانية EDA أدوات 7.1

Pandas Profiling (ydata-profiling)

توزيعات، قيم — Dataset كامل تلقائياً بكل المعلومات عن HTML يعمل تقرير Pandas Profiling بدل ما تكتب كل تحليل يدوياً، الـ كل حاجة، correlations، outliers، مفقودة

```
from ydata_profiling import ProfileReport

profile = ProfileReport(df, title='تقرير تحليل البيانات')
profile.to_file('eda_report.html') # تفاعلي HTML يحفظ تقرير
```

Sweetviz

ده مفيد جداً قبل التدريب — (Train vs Test مثلاً) أداة ثانية رائعة، تتميز بأنها بتقارن بين مجموعتين من البيانات

```
import sweetviz as sv

report = sv.analyze(df)
report.show_html('sweetviz_report.html')

# مقارنة Train vs Test
compare_report = sv.compare([train_df, 'Train'], [test_df, 'Test'])
compare_report.show_html('comparison.html')
```

Python كاملة بـ EDA Pipeline 7.2

متكاملة تجمع كل ما تعلمناه pipeline دلوقتي هنبني

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

def complete_eda_pipeline(df, target=None):
    """شاملة EDA Pipeline"""

    # --- 1. المعلومات الأساسية ---
    print('الشكل:', df.shape)
    print('أنواع البيانات:')
    print(df.dtypes.value_counts())
```

```

# --- 2. القيم المفقودة ---
missing_pct = (df.isnull().sum() / len(df) * 100).round(2)
missing_df = pd.DataFrame({'نسبة المفقودة': missing_pct})
print('\nأعمدة بها قيم مفقودة:')
print(missing_df[missing_df0] > 0['نسبة المفقودة'])

# --- 3. الملخص الإحصائي ---
print('\nملخص إحصائي:')
print(df.describe())

# --- 4. اكتشاف الـ Outliers ---
numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    outliers_count = ((df[col] < Q1-1.5*IQR) | (df[col] > Q3+1.5*IQR)).sum()
    if outliers_count > 0:
        print(f'{col}: {outliers_count} outliers  
{outliers_count/len(df)*100:.1f}%')

# --- 5. Correlation ---
if target and target in df.columns:
    corr = df.corr()[target].sort_values(ascending=False)
    print(f'\nارتباط Features بـ {target}:')
    print(corr)

return df

```

ML و AI في Descriptive Statistics الفصل الثامن: تطبيقات

8.1 Computer Vision Statistics

classes، الإحصاء الوصفي يُستخدم لفهم بيانات الصور قبل التدريب. بتحليل توزيع الألوان، حجم الصور، نسبة الـ Computer Vision في الـ وغيرها.

مثال عملي: في مشروع اكتشاف أمراض الصدر من الأشعة، لازم تعرف

- (class imbalance) كام % من الصور فيها مرض
- ما متوسط درجة سطوع الصور؟
- ما التباين في أحجام الصور؟

```
# Class Distribution تحليل
class_counts = df['diagnosis'].value_counts()
class_pct = class_counts / len(df) * 100
print('Classes توزيع الـ')
print(class_pct)

# techniques بنستخدم، imbalance لو في
# SMOTE, Class Weights, أو Oversampling
```

8.2 NLP Statistics

الإحصاء الوصفي ببساعدك تفهم البيانات النصية، NLP في الـ

```
# إحصاءات النصوص
df['text_length'] = df['text'].str.len()
df['word_count'] = df['text'].str.split().str.len()
df['unique_words'] = df['text'].apply(lambda x: len(set(x.split())))

print(df[['text_length', 'word_count', 'unique_words']].describe())
```

8.3 Anomaly Detection

، بتنظف البيانات من الشواذ Outlier Analysis الفرق: في الـ Outliers هو تطبيق مباشر لكل ما تعلمناه عن الـ Anomaly Detection الـ الشواذ نفسها هي اللي بتحاول تكتشفها Anomaly Detection في الـ

تطبيقات: اكتشاف الاحتيال المالي، اكتشاف أعطال المعدات، الأمن السيبراني

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler

# تجهيز البيانات
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```

# Anomaly Detection بناء موديل الـ
model = IsolationForest(contamination=0.05, random_state=42)
predictions = model.fit_predict(X_scaled)

# -1 = Anomaly، 1 = طبيعي
anomalies = X[predictions == -1]
print(f'الـ عدد Anomalies: {len(anomalies)}')

```

8.4 Monitoring ML Models

وقت ما توزيع — Data Drift لازم تراقبه باستمرار. الإحصاء الوصفي هنا يساعد في اكتشاف، Production بعد ما نشر الموديل في الـ البيانات الجديدة يختلف عن البيانات اللي اتدرب عليها الموديل.

```

# Training vs Production مقارنة توزيع
from scipy import stats

# KS Test - لمقارنة توزيعين
stat, p_value = stats.ks_2samp(train_data['feature'], prod_data['feature'])

if p_value < 0.05:
    print('مكتشف! - لازم تعيد تدريب الموديل Data Drift: تحذير')
else:
    print('التوزيع طبيعي - الموديل شغال كويس')

```

الفصل التاسع: مشاريع تطبيقية

مشروع 1 — تحليل بيانات الطلاب 9.1

كاملة لبيانات طلاب: درجاتهم، حضورهم، خلفيتهم الاجتماعية EDA pipeline هبني

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

# تحميل البيانات (مثال)
df = pd.read_csv('students.csv')

# فهم البيانات 1.
print('شكل البيانات:', df.shape)
print(df.describe())

# تحليل توزيع الدرجات 2.
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
df['grade'].hist(bins=20, ax=axes[0], title='توزيع الدرجات')
sns.boxplot(data=df, y='grade', ax=axes[1])
plt.tight_layout()
plt.show()

# العلاقة بين الحضور والدرجات 3.
sns.scatterplot(data=df, x='attendance', y='grade', hue='gender')
corr = df['attendance'].corr(df['grade'])
print(f'الارتباط بين الحضور والدرجات: {corr:.3f}')

# اكتشاف الطلاب غير الملتزمين 4. (Outliers)
Q1 = df['grade'].quantile(0.25)
Q3 = df['grade'].quantile(0.75)
IQR = Q3 - Q1
at_risk = df[df['grade'] < Q1 - 1.5 * IQR]
print(f'طلاب في خطر: {len(at_risk)}')
```

مشروع 2 — تحليل أسعار المنازل 9.2

زي المساحة، الغرف، الموقع، العمر features تحليل بيانات أسعار المنازل يشمل

```
df = pd.read_csv('house_prices.csv')

# 1. التوزيع - السعر عادة Skewed
print(f'Skewness للسعر: {df["price"].skew():.2f}')

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```



```

df['price'].hist(bins=50, ax=axes[0])
axes[0].set_title('Skewed - توزيع السعر')
np.log1p(df['price']).hist(bins=50, ax=axes[1])
axes[1].set_title('السعر - أكثر تماثلاً Log توزيع')
plt.show()

# 2. Correlation مع السعر
numeric_cols = df.select_dtypes(include=np.number).columns
corr_with_price = df[numeric_cols].corr()['price'].sort_values(ascending=False)
print('العوامل الأكثر تأثيراً على السعر:')
print(corr_with_price.head(10))

# 3. Outlier Detection في الأسعار
from sklearn.ensemble import IsolationForest
features = ['area', 'rooms', 'age', 'price']
iso_forest = IsolationForest(contamination=0.02)
df['is_outlier'] = iso_forest.fit_predict(df[features])
print(f'منازل شاذة: {(df["is_outlier"]==-1).sum()}')
```

للإحصاء الوصفي Dashboard مشروع 3 — بناء 9.3

تحتها فيه Dataset تلقائياً على أي EDA يعمل Dashboard الحقيقية، تحتاج AI في مشاريع الـ

```

def statistical_dashboard(df, target_col=None):
    """شامل للإحصاء الوصفي Dashboard"""

    numeric = df.select_dtypes(include=np.number)
    categorical = df.select_dtypes(exclude=np.number)

    n_numeric = len(numeric.columns)

    fig = plt.figure(figsize=(20, 5 * n_numeric))

    for i, col in enumerate(numeric.columns, 1):
        ax1 = fig.add_subplot(n_numeric, 3, (i-1)*3 + 1)
        numeric[col].hist(bins=30, ax=ax1)
        ax1.set_title(f'توزيع {col}')

        ax2 = fig.add_subplot(n_numeric, 3, (i-1)*3 + 2)
        numeric[col].plot.box(ax=ax2)
        ax2.set_title(f'Box Plot - {col}')

        ax3 = fig.add_subplot(n_numeric, 3, (i-1)*3 + 3)
        stats_text = f'Mean: {numeric[col].mean():.2f}\n'
        stats_text += f'Median: {numeric[col].median():.2f}\n'
        stats_text += f'Std: {numeric[col].std():.2f}\n'
        stats_text += f'Skew: {numeric[col].skew():.2f}'
        ax3.text(0.1, 0.5, stats_text, transform=ax3.transAxes, fontsize=12)
        ax3.axis('off')
```

```
plt.tight_layout()
plt.savefig('dashboard.png', dpi=150, bbox_inches='tight')
plt.show()
```

9.4 4 مشروع — EDA Project كامل (AI Dataset Analysis)

بكل الخطوات من البداية للنهاية — Heart Disease Dataset — Kaggle كاملة من Dataset هنحل

```
# خطوة 1: تحميل وفهم البيانات
df = pd.read_csv('heart_disease.csv')
print(df.head())
print(df.info())

# خطوة 2: تنظيف البيانات
print('قيم مفقودة:')
print(df.isnull().sum())
df = df.drop_duplicates()

# خطوة 3: اكتشاف Outliers
numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    Q1, Q3 = df[col].quantile([0.25, 0.75])
    IQR = Q3 - Q1
    mask = (df[col] < Q1-1.5*IQR) | (df[col] > Q3+1.5*IQR)
    print(f'{col}: {mask.sum()} outliers')

# خطوة 4: Correlation Analysis
plt.figure(figsize=(12, 10))
sns.heatmap(df.corr(), annot=True, fmt='.2f', cmap='coolwarm', center=0)
plt.title('مصفوفة الارتباط')
plt.savefig('correlation.png', dpi=150)

# خطوة 5: تحليل العلاقات مع الـ Target
target = 'heart_disease'
for col in numeric_cols:
    if col != target:
        group0 = df[df[target]==0][col]
        group1 = df[df[target]==1][col]
        t_stat, p_val = stats.ttest_ind(group0, group1)
        if p_val < 0.05:
            print(f'{col}: فرق معنوي (p={p_val:.4f})')
```

ملاحظة ختامية

كل مشروع من المشاريع دي هو تطبيق حقيقي للمفاهيم النظرية اللي اتعلمناها في الفصول السابقة. الأهم إنك تفهم العلاقة بين كل خطوة ولية بتعملها — مش بس 'إزاي'. لما تسأل دايماً 'ليه؟' قبل 'إزاي؟' — هتفهم فعلاً.

ملخص المفاهيم الأساسية

AI ارتباطه بالـ	متى تستخدمه	المفهوم
تنظيف بيانات التدريب	Outliers بيانات غير طبيعية أو فيها	IQR Method
مسبقاً Feature Scaling	بيانات تتبع التوزيع الطبيعي	Z-Score
للموديل Features تجهيز	Symmetric بيانات، MCAR	Mean Imputation
بيانات مالية ورواتب	Outliers أو فيها Skewed بيانات	Median Imputation
Categorical Features تجهيز	(Nominal) فئات بدون ترتيب	One-Hot Encoding
Small/Medium/Large بيانات مثل	(Ordinal) فئات مترتبة	Label Encoding
NLP classification	تحليل نصوص واستخراج أهمية الكلمات	TF-IDF
Recommendation Systems	vectors مقارنة وثائق أو	Cosine Similarity
تنبؤ بالمبيعات والأسعار	Time Series تخفيف ضوضاء	Moving Average
Fraud Detection	اكتشاف شواذ في بيانات متعددة الأبعاد	Isolation Forest